

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Engineering (CSE)

MID SEMESTER EXAMINATION
DURATION: 2 HOURS

SUMMER SEMESTER, 2023-2024
FULL MARKS: 120

CSE 4205: Digital Logic Design

Programmable calculators are not allowed. Do not write anything on the question paper.

Answer **all 4 (four)** questions. Figures in the right margin indicate full marks of questions with corresponding COs and POs in parentheses.

1. Assume that Excess-5 is a coding scheme where a decimal number is represented by adding $(5)_{10}$ to its binary equivalent. For example, $(0)_{10}$ is represented as $(0101)_{XS-5}$ and decimal $(9)_{10}$ is $(1110)_{XS-5}$.

The Excess-5 encoder accepts only valid codes representing decimal digits from $(0)_{10}$ to $(9)_{10}$. All other input combinations are considered invalid and their corresponding outputs are treated as don't care conditions.

- | | |
|---|----------------------|
| a) Draw the truth table for the Binary-to-Excess-5 encoder based on the information provided. Clearly label all input and output columns, and appropriately handle the specified conditions for invalid inputs. | 4
(CO1)
(PO1) |
| b) Derive the simplified expressions for each of the output bits using K-map. | 16
(CO3)
(PO2) |
| c) Construct a two-level logic circuit for each output bit using only NAND gates. Use NAND gates to generate complemented input variables as needed (this preprocessing is not considered part of the two-level implementation). Draw the logic circuits for all output bits together in a single diagram, using the same set of input variables. | 7
(CO2)
(PO2) |
| d) Define what a self-complementing code is. Excess-3 is known to be a self-complementing code, but is Excess-5 also a self-complementing code? If yes, provide an example to illustrate why. If not, present a counterexample to demonstrate why it does not satisfy the property. | 3
(CO1)
(PO1) |

2. a) In digital communication, a checksum is a simple error-detection method used to verify data integrity. The sender transmits data along with a checksum (typically the sum of the values). The receiver then adds the received data and compares the result to the checksum. If the difference is zero, the data is assumed to be unaltered.

Alice intends to send five numbers to Bob, along with their checksum. However, due to transmission loss, only four of the numbers and the checksum reach him. Although a checksum is typically used for error detection rather than error correction or recovery, Bob assumes that the received numbers and the checksum remained intact during transmission and attempts to use them to recover the missing number.

The four numbers and the checksum that reached Bob are:

$$numbers = (785)_{10}, (102)_{10}, (637)_{10}, (954)_{10}$$

$$checksum = (10630)_7$$

- | | |
|--|---------------------|
| i. Convert the four numbers into their corresponding Binary-Coded Decimal (BCD) representations. | 4
(CO1)
(PO1) |
|--|---------------------|

- ii. Find the sum of the four numbers obtained in Question 2.a)i using Binary-Coded Decimal (BCD) addition. 8
(CO2)
(PO2)
- iii. Convert the sum obtained in Question 2.a)ii into a base 7 number and subtract it from the checksum using Radix complement (7's complement) to compute the number that was lost during transmission (use a 6-digit register, where the most significant bit (MSB) or the leftmost digit, denotes the sign bit). 2 + 8
(CO1)
(PO1)
- b) Can the checksum result be zero even if the transmitted numbers have been modified? In other words, can a change go undetected when using a checksum? Briefly explain your reasoning with an example. 5
(CO1)
(PO1)
- c) Even though binary and BCD representations of decimal digits are quite similar, why does BCD addition require an adjustment step that binary addition does not? 3
(CO1)
(PO1)
3. a) Simplify the following boolean expression using Quine-McCluskey (Tabular) method. 15
(CO5)
(PO1)
- $$F(A, B, C, D) = \prod M(3, 5, 6, 9, 12) \cdot \prod d(2, 10, 14)$$
- While solving using the Tabular Method, clearly state the prime implicants and essential prime implicants.
- b) Using cut and try method, prove the following - 5 + 5
(CO2)
(PO2)
- i. $xy + x'z + yz = xy + x'z$
- ii. $xy'z + x'yz + xy = xy + yz + zx$
- c) Can parity always successfully detect an error? If yes, briefly justify your answer, if not, provide a counterexample. 2 + 3
(CO1)
(PO1)
- A system uses odd parity for error detection. Which of the following bit patterns (including the parity bit) indicates an error?
- i. 1101010
- ii. 1011101
4. a) What is the advantage of using a lookahead carry adder compared to a ripple carry adder? 5 + 10
(CO5)
(PO1)
- Provide the necessary equations for carry generation and propagation to support your explanation. Also, compare their circuit or block diagrams to illustrate the difference.
- b) Design a 3-bit Full Adder-Subtractor circuit and explain its working mechanism, including how it functions as both an adder and a subtractor. 5 + 5
(CO5)
(PO3)
- c) Is the circuit shown in Figure 1 a sequential or combinational circuit? Justify your answer. 5
(CO1)
(PO1)

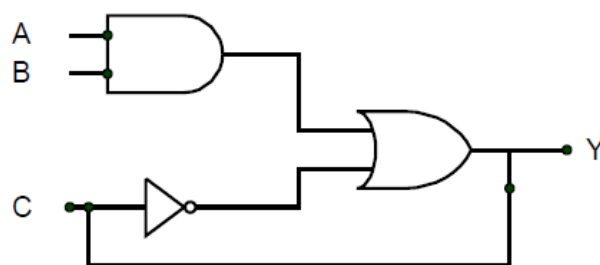


Figure 1: A boolean circuit

1. Assume that Excess-5 is a coding scheme where a decimal number is represented by adding $(5)_{10}$ to its binary equivalent. For example, $(0)_{10}$ is represented as $(0101)_{XS-5}$ and decimal $(9)_{10}$ is $(1110)_{XS-5}$.

The Excess-5 encoder accepts only valid codes representing decimal digits from $(0)_{10}$ to $(9)_{10}$. All other input combinations are considered invalid and their corresponding outputs are treated as don't care conditions.

- a) Draw the truth table for the Binary-to-Excess-5 encoder based on the information provided. Clearly label all input and output columns, and appropriately handle the specified conditions for invalid inputs.

4
(CO1)
(PO1)

Solution:

Decimal	Input (Binary)				Output (Excess-5)			
Value	A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	1	0	1
1	0	0	0	1	0	1	1	0
2	0	0	1	0	0	1	1	1
3	0	0	1	1	1	0	0	0
4	0	1	0	0	1	0	0	1
5	0	1	0	1	1	0	1	0
6	0	1	1	0	1	0	1	1
7	0	1	1	1	1	1	0	0
8	1	0	0	0	1	1	0	1
9	1	0	0	1	1	1	1	0
10	1	0	1	0	X	X	X	X
11	1	0	1	1	X	X	X	X
12	1	1	0	0	X	X	X	X
13	1	1	0	1	X	X	X	X
14	1	1	1	0	X	X	X	X
15	1	1	1	1	X	X	X	X

Invalid

Don't care

Solution:

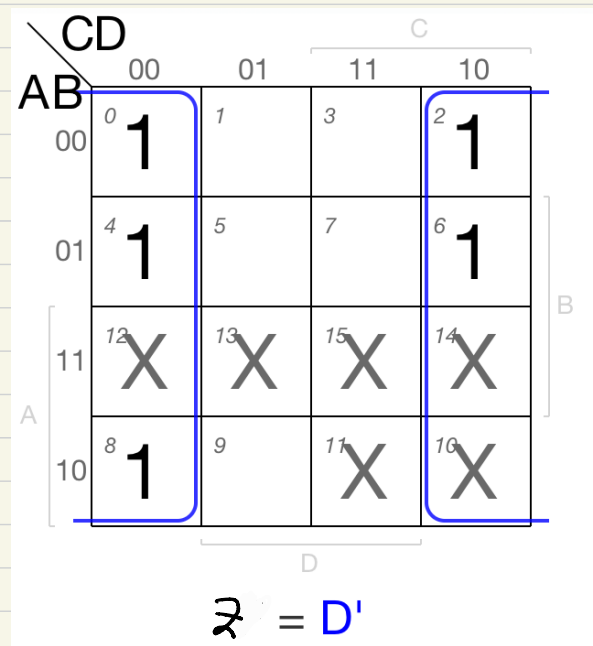
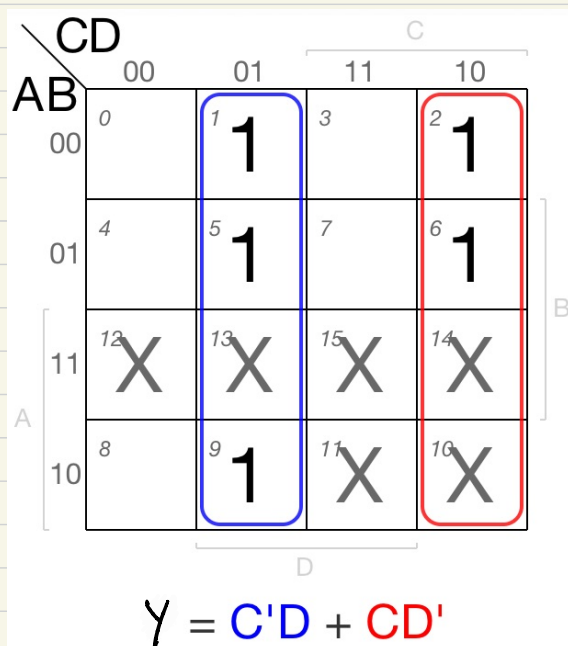
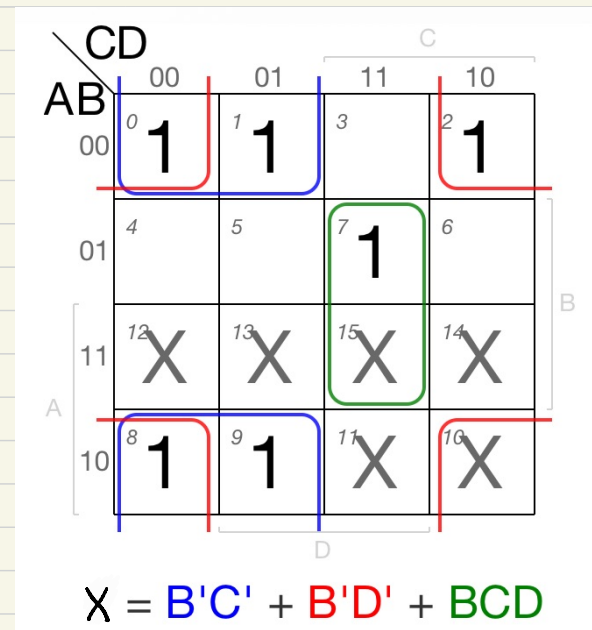
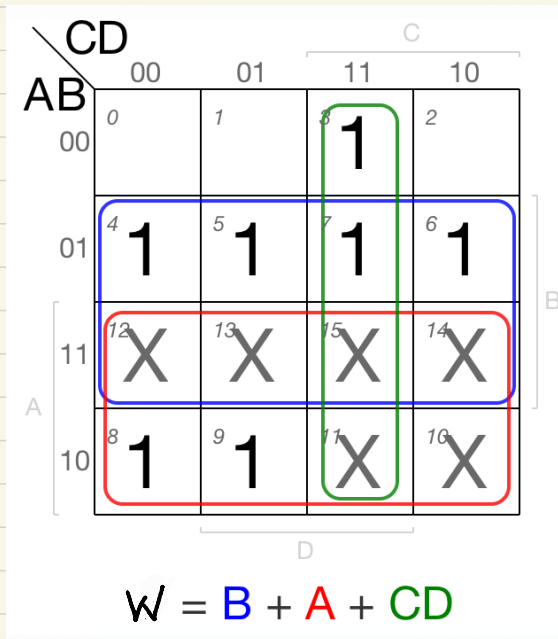
Rubrics:

- 0.25 points each (times 16) for each input - output combination of the truth table (including don't care and invalid conditons).

b) Derive the simplified expressions for each of the output bits using K-map.

16
(CO3)
(PO2)

Solution:



Solution:

Rubrics:

For each Karnaugh map (4 in total):

- 1 point for correctly constructing the K-map.
- 1 point for correctly handling the don't care conditions.
- 1 point for correctly identifying valid groupings.
- 1 point for correctly determining the Essential Prime Implicants representing the groups.

c) Construct a two-level logic circuit for each output bit using only NAND gates. Use NAND gates to generate complemented input variables as needed (this preprocessing is not considered part of the two-level implementation). Draw the logic circuits for all output bits together in a single diagram, using the same set of input variables.

7
(CO2)
(PO2)

Solution:

$$W = B + A + CD$$

$$= \overline{\overline{B} \cdot \overline{A} \cdot \overline{CD}}$$

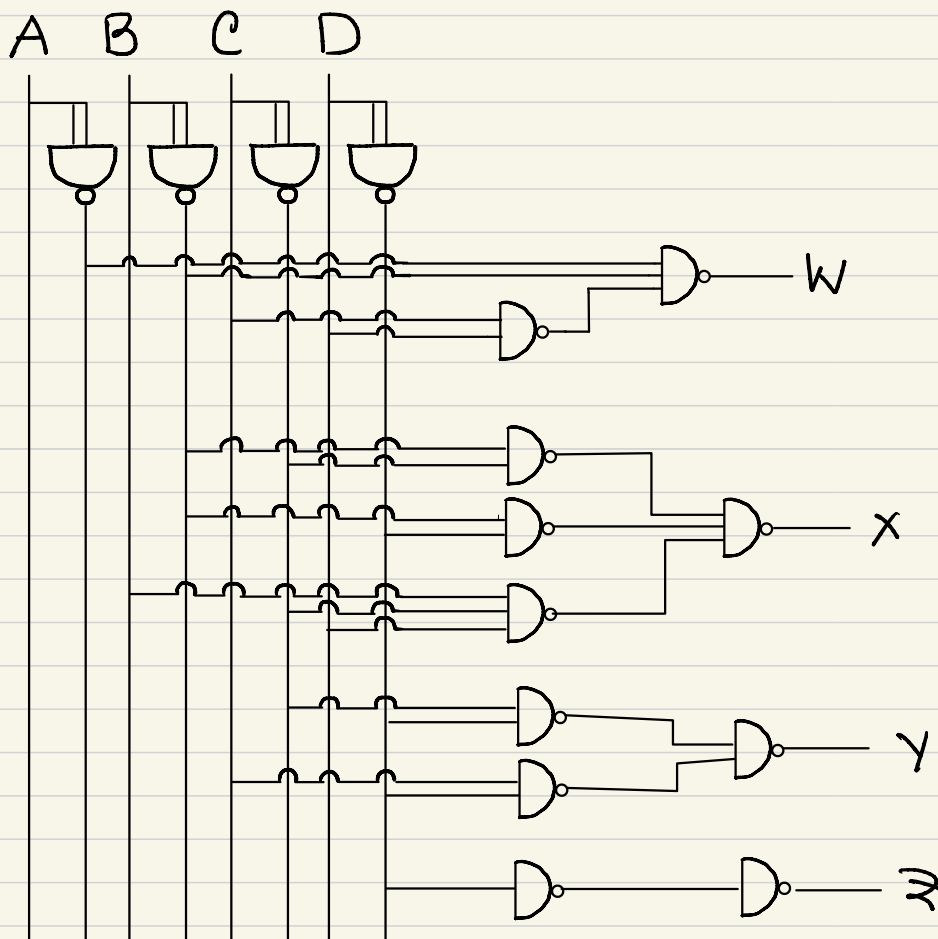
$$X = \overline{B} \overline{C} + \overline{B} \overline{D} + BCD$$

$$= \overline{\overline{\overline{B} \overline{C}} \cdot \overline{\overline{B} \overline{D}} \cdot \overline{BCD}}$$

$$Y = \overline{C} D + C \overline{D}$$

$$= \overline{\overline{C} D \cdot C \overline{D}}$$

$$Z = \overline{D}$$



Solution:

Rubrics:

- 1 point for correctly constructing the original and complemented input using NAND gate.
- 0.75 point each (times 4) for correctly deriving the equation compatible with NAND implementation format.
- 0.75 point each (times 4) for correctly drawing the circuit.

d) Define what a self-complementing code is. Excess-3 is known to be a self-complementing code, but is Excess-5 also a self-complementing code? If yes, provide an example to illustrate why. If not, present a counterexample to demonstrate why it does not satisfy the property.

3
(CO1)
(PO1)

Solution:

Self Complementing Code

If code of the complement of a number is same as the complement of that number's code.

<u>Decimal</u>	<u>Excess-3</u>	<u>Excess-5</u>
2	0 1 0 1	0 1 1 1
7	1 0 1 0	1 1 1 1

9's or (10-1)'s complement

1's or (2-1)'s complement

So, NO, Excess-5 is not self complementing.

Solution:

Rubrics:

- 1 point for correctly defining self-complementing code.
- 1 point for mentioning that Excess-5 is not a self complementing code.
- 1 point for justification or counterexample.

2. a) In digital communication, a checksum is a simple error-detection method used to verify data integrity. The sender transmits data along with a checksum (typically the sum of the values). The receiver then adds the received data and compares the result to the checksum. If the difference is zero, the data is assumed to be unaltered.

Alice intends to send five numbers to Bob, along with their checksum. However, due to transmission loss, only four of the numbers and the checksum reach him. Although a checksum is typically used for error detection rather than error correction or recovery, Bob assumes that the received numbers and the checksum remained intact during transmission and attempts to use them to recover the missing number.

The four numbers and the checksum that reached Bob are:

$$\text{numbers} = (785)_{10}, (102)_{10}, (637)_{10}, (954)_{10}$$

$$\text{checksum} = (10630)_7$$

- i. Convert the four numbers into their corresponding Binary-Coded Decimal (BCD) representations.

4
(CO1)
(PO1)

Solution :

i)

Decimal Number	Digits	BCD Representation
785	7, 8, 5	0111 1000 0101
102	1, 0, 2	0001 0000 0010
637	6, 3, 7	0110 0011 0111
954	9, 5, 4	1001 0101 0100

Solution:

Rubrics:

- 1 point for each (times 4) correct BCD representation.

(ii) Solution:

$$\begin{array}{r}
 (785)_{10} \\
 + (102)_{10} \\
 \hline
 (887)_{10}
 \end{array}
 \quad
 \begin{array}{r}
 0111 \quad 1000 \quad 0101 \\
 + 0001 \quad 0000 \quad 0010 \\
 \hline
 1000 \quad 1000 \quad 0111
 \end{array}$$

$$\begin{array}{r}
 (637)_{10} \\
 + (954)_{10} \\
 \hline
 (1591)_{10}
 \end{array}
 \quad
 \begin{array}{r}
 0110 \quad 0011 \quad 0111 \\
 + 1001 \quad 0101 \quad 0100 \\
 \hline
 1111 \quad 1000 \quad 1011 \\
 + 0110 \quad + \quad 1 \quad + 0110 \\
 \hline
 1 \quad 0101 \quad 1001 \quad 0001
 \end{array}$$

$$\begin{array}{r}
 (1591)_{10} \\
 + (887)_{10} \\
 \hline
 (2478)_{10}
 \end{array}
 \quad
 \begin{array}{r}
 \overset{1}{1} \quad 0101 \quad 1001 \quad 0001 \\
 + 1000 \quad 1000 \quad 0111 \\
 \hline
 1 \quad 1110 \quad 10001 \quad 1000 \\
 + 0110 \quad + 0110 \\
 \hline
 10 \quad 0100 \quad 0111 \quad 1000
 \end{array}$$

Ans: $(0010 \ 0100 \ 0111 \ 1000)_{BCD}$

Solution:

Rubrics:

- 1 point for each (times 3) correct BCD addition.
- 2 points for correctly handling the case when the sum of two digits exceeds 9.

(iii) Solution:

$$(0010 \ 0100 \ 0111 \ 1000)_{BCD} = (2478)_{10}$$

Base 10 to Base 7

$$\begin{array}{r|l} 7 & 2478 \\ \hline 7 & 354 \quad - \quad 0 \\ \hline 7 & 50 \quad - \quad 4 \\ \hline 7 & 7 \quad - \quad 1 \\ \hline 7 & 1 \quad - \quad 0 \\ \hline & 0 \quad - \quad 1 \end{array}$$

$$(2478)_{10} = (10140)_7$$

$$\begin{aligned} \text{Missing number} &= (10630)_7 - (10140)_7 \\ &= (10630)_7 + (-10140)_7 \end{aligned}$$

$$\begin{array}{rcccccc} (10630)_7 & & 0 & 1 & 0 & 6 & 3 & 0 \\ 7\text{'s Complement of } (10140)_7 & (+) & 6 & 5 & 6 & 5 & 3 & 0 \\ \hline \end{array}$$

$$\begin{array}{r|l} \overset{\text{overflow}}{\cancel{1}} & 0 \quad 0 \quad 0 \quad 4 \quad 6 \quad 0 \end{array}$$

$$\begin{aligned} \text{Missing number} &= (460)_7 \text{ or } (4 \times 7^2 + 6 \times 7)_{10} \\ &= (238)_{10} \end{aligned}$$

(Ans.)

Solution:**Rubrics:**

- 2 points for correctly converting the BCD sum to base 7 number.
- 1 point for considering 6 digit register.
- 1 point for writing subtraction in form of addition.
- 2 points for complementing the number using 7's complement.
- 3 points for correct subtraction.
- 1 point for correct answer.

b) Can the checksum result be zero even if the transmitted numbers have been modified? In other words, can a change go undetected when using a checksum? Briefly explain your reasoning with an example.

5
(CO1)
(PO1)

Solution:

YES.

1. If switching / swapping of digits occurs between numbers in a specific position.
2. If the sum remains unchanged.

Example:

		<u>Case I</u>		<u>Case II</u>	
data 1	253	data 1	223	data 1	258 (+5)
data 2	322	data 2	352	data 2	317 (-5)
Checksum	575	Checksum	575	Checksum	575

In either case the change will get undetected, as the checksum value remains same.

Solution:

Rubrics:

- 1 point for mentioning that it might go undetected.
- 2 points each (times 2) for justification and example.

c) Even though binary and BCD representations of decimal digits are quite similar, why does BCD addition require an adjustment step that binary addition does not?

3
(CO1)
(PO1)

Solution :

Binary-Coded Decimal (BCD) uses 4-bit binary numbers to represent only decimal digits 0 through 9 (i.e., 0000 to 1001). However, regular binary addition does not restrict results to this range. When adding two BCD digits, the binary sum can exceed 1001 (9 in decimal), resulting in an invalid BCD code.

To correct this and bring the result back into the valid BCD range, an adjustment is made: if the sum exceeds 9 or if a carry is generated, we add 6 (0110 in binary) to the result. This adjustment also ensures that a carry is propagated to the next BCD digit correctly, preserving the integrity of the decimal representation.

Solution:

Rubrics:

- 1 point for correctly stating that BCD addition requires adjustment because BCD is not a pure binary system.
- 1 point for explaining that the adjustment is needed when the binary sum exceeds 9 (1001 in binary), which is the maximum valid BCD digit.
- 1 point for mentioning that an addition of 6 (0110) is required to correct invalid BCD results and carry over to the next digit.

3. a) Simplify the following boolean expression using Quine-McCluskey (Tabular) method.

15
(CO5)
(PO1)

$$F(A, B, C, D) = \prod M(3, 5, 6, 9, 12) \cdot \prod d(2, 10, 14)$$

While solving using the Tabular Method, clearly state the prime implicants and essential prime implicants.

Solution :

$$F(A, B, C, D) = \sum m(0, 1, 4, 7, 8, 11, 13, 15) + \sum d(2, 10, 14)$$

1

NEW FUNCTION

3. Finding Prime Implicants		
Group	Decimal	Binary
0	0, 8	-000 ✓
0	0, 4	0-00 *
0	0, 2	00-0 ✓
0	0, 1	000- *
1	10, 8	10-0 ✓
1	10, 2	-010 ✓
2	10, 14	1-10 ✓
2	10, 11	101- ✓
3	15, 7	-111 *
3	11, 15	1-11 ✓
3	13, 15	11-1 *
3	14, 15	111- ✓

2. Finding Prime Implicants		
Group	Decimal	Binary
0	0	0000 ✓
1	1	0001 ✓
1	4	0100 ✓
1	8	1000 ✓
1	2	0010 ✓
2	10	1010 ✓
3	7	0111 ✓
3	11	1011 ✓
3	13	1101 ✓
3	14	1110 ✓
4	15	1111 ✓

4. Finding Prime Implicants		
Group	Decimal	Binary
0	0, 10, 2, 8	-0-0 *
1	10, 11, 14, 15	1-1- *

The Prime Implicants Are
A'C'D', A'B'C', BCD, ABD, B'D', AC

5

6. Prime Implicants Chart									
Minterms	0	1	11	13	15	4	7	8	Cost
A'C'D'	X					X			4
A'B'C'	X	X							4
BCD					X		X		4
ABD				X	X				4
B'D'	X							X	3
AC			X		X				3

7. Finding Unique Minterms									
Minterms	0	1	11	13	15	4	7	8	Cost
A'C'D'	X					X			4
A'B'C'	X	X							4
BCD					X		X		4
ABD				X	X				4
B'D'	X							X	3
AC			X		X				3

8

The Essential Implicants Are
A'C'D', A'B'C', BCD, ABD, B'D', AC

9

Possible Function Minimizations
 $F = A'C'D' + A'B'C' + BCD + ABD + B'D' + AC$

Solution:

Rubrics:

- 1 point for correctly converting maxterms and dont care terms to their binary form.
- 1 point for grouping them based on the number of 1s.
- 3+3+2 points for performing all valid pairwise combinations to generate prime implicants (including use of dash notation and proper grouping).
- 3 points for correctly identifying and listing all prime implicants.
- 2 marks Accurately determining essential prime implicants using a prime implicant chart and final correct simplified Boolean expression in Product of Sums (POS) form.

b) Using cut and try method, prove the following -

5 + 5
(CO2)
(PO2)

i. $xy + x'z + yz = xy + x'z$

ii. $xy'z + x'yz + xy = xy + yz + zx$

Solution :

$$\begin{aligned} \text{i) L.H.S} &= xy + \bar{x}z + yz \\ &= xy + \bar{x}z + (\bar{x} + x)yz \\ &= xy + \bar{x}z + \bar{x}yz + xyz \\ &= xy + xyz + \bar{x}z + \bar{x}yz \\ &= xy(1+z) + \bar{x}z(1+y) \\ &= xy + \bar{x}z \\ &= \text{R.H.S} \end{aligned}$$

$$\begin{aligned} \text{ii) R.H.S} &= x\bar{y}z + \bar{x}yz + xy \\ &= x\bar{y}z + \bar{x}yz + xy(z + \bar{z}) \\ &= x\bar{y}z + \bar{x}yz + xyz + xy\bar{z} \\ &= x\bar{y}z + xyz + \bar{x}yz + xyz + xy\bar{z} + xy\bar{z} \\ &= x(\bar{y} + y)z + (\bar{x} + x)yz + xy(\bar{z} + z) \\ &= xz + yz + xy \\ &= \text{R.H.S} \end{aligned}$$

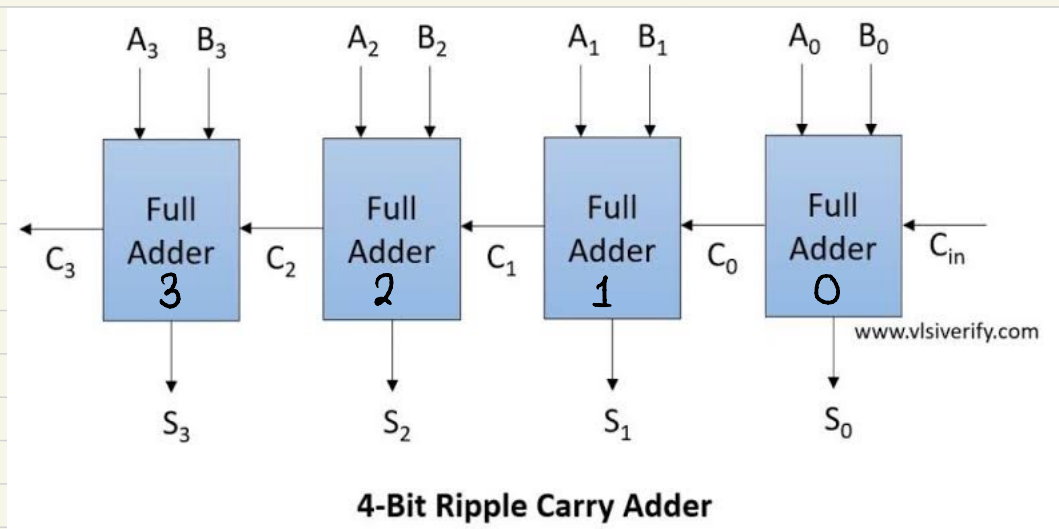
Solution:

Rubrics:

For each subtask

- 2 point for correctly identifying and grouping terms to apply Boolean identities.
- 2 point for correctly applying Boolean algebra laws (e.g., distributive, absorption, De Morgans).
- 1 point for logically deriving the right-hand side from the left-hand side.

4. a) What is the advantage of using a lookahead carry adder compared to a ripple carry adder? 5 + 10
Provide the necessary equations for carry generation and propagation to support your explanation. Also, compare their circuit or block diagrams to illustrate the difference. (CO5) (PO1)



In Ripple Carry Adder, intermediate carries are propagated to the next Full Adder, which means F/A 3 needs to wait for F/A 2 to generate C_2 , F/A 2 needs to wait for F/A 1 and so on. This phenomenon is known as **propagational delay**.

Lookahead Carry Adder solves this problem by pre-calculating intermediate carries using available inputs, so there is no delay.

$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = P_i C_i + G_i$$

$$S_0 = P_0 \oplus C_0$$

$$C_1 = P_0 C_0 + G_0$$

$$S_1 = P_1 \oplus C_1 = P_1 \oplus (P_0 C_0 + G_0)$$

$$\begin{aligned} C_2 &= P_1 C_1 + G_1 \\ &= P_1 (P_0 C_0 + G_0) + G_1 \\ &= P_1 P_0 C_0 + P_1 G_0 + G_1 \end{aligned}$$

$$S_2 = P_2 \oplus C_2 = P_2 \oplus (P_1 P_0 C_0 + P_1 G_0 + G_1)$$

$$\begin{aligned} C_3 &= P_2 C_2 + G_2 \\ &= P_2 (P_1 P_0 C_0 + P_1 G_0 + G_1) + G_2 \\ &= P_2 P_1 P_0 C_0 + P_2 P_1 G_0 + P_2 G_1 + G_2 \end{aligned}$$

- - - - -

All expressions depends on already available values $A(A_3 \sim A_0)$, $B(B_3 \sim B_0)$ and C_0 .

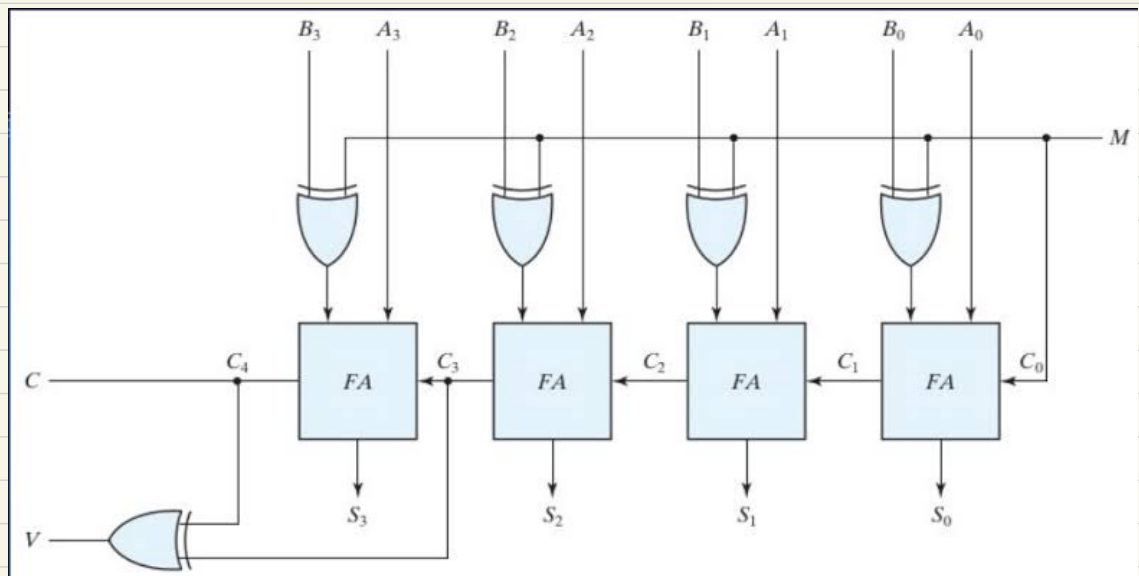
Doesn't rely on intermediate values.

Solution:**Rubrics:**

- 3 points for correctly stating the advantage of lookahead carry adder over ripple carry adder (e.g., faster carry calculation reducing delay).
- 4 points for providing correct equations for carry generation ($G_i = A_i B_i$) and propagation ($P_i = A_i + B_i$) and explaining their role.
- 3 points for explaining how these equations enable lookahead carry calculation.
- 2.5 points each (times) for providing a clear comparison between the circuit/block diagrams of ripple carry and lookahead carry adders.

b) Design a 3-bit Full Adder-Subtractor circuit and explain its working mechanism, including how it functions as both an adder and a subtractor.

5 + 5
(CO5)
(MCQ)



Adder ($M = 0$)

$$B \oplus M = B$$

$$1 \oplus 0 = 1$$

$$0 \oplus 0 = 0$$

$$\text{ADD}(B, M) = B$$

$$(M = 0)$$

Subtractor ($M = 1$)

$$B \oplus M = \bar{B}$$

$$\left. \begin{array}{l} 1 \oplus 1 = 0 \\ 0 \oplus 1 = 1 \end{array} \right\} \begin{array}{l} \text{1's} \\ \text{complement} \end{array}$$

$$\text{ADD}(\bar{B}, M) = 2\text{'s complement of } B$$

$$(M = 1) \quad B$$

Solution:**Rubrics:**

- 1 point for clearly explaining the control input (e.g., mode or M) that selects between addition and subtraction.
- 1 point for correctly describing how subtraction is implemented using 2's complement (inverting B and adding 1).
- 1 point for providing the logic-level design of a 1-bit full adder-subtractor and extending it to 3-bit.
- 1 point for correct connection of XOR gates for conditional inversion of B inputs.
- 1 point for explaining the carry-in setup for subtraction (initial carry-in = 1).
- 2 point for including a block diagram or circuit diagram of the 3-bit adder-subtractor.
- 2 point for explaining how the circuit switches between addition and subtraction modes.
- 1 point for illustrating the working with an example (e.g., 3-bit addition and subtraction cases).

c) Is the circuit shown in Figure 1 a sequential or combinational circuit? Justify your answer.

5
(CO1)
(PO1)

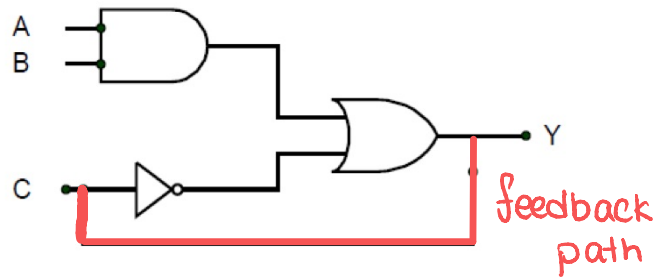


Figure 1: A boolean circuit

Existence of feedback path } Sequential circuit
Input depends on output

Solution:

Rubrics:

- 1 point for correctly defining a combinational circuit (output depends only on present inputs).
- 1 point for correctly defining a sequential circuit (output depends on present inputs and past history via feedback/memory).
- 1 point for observing and correctly identifying the presence of a feedback path in the given circuit (Figure 1).
- 1 point for concluding that the circuit is sequential based on the presence of feedback.
- 1 point for clear justification and explanation linking feedback to memory and sequential behavior.

and past history via feedback/memory).

- 1 point for observing and correctly identifying the presence of a feedback path in the given circuit (Figure 1).
- 1 point for concluding that the circuit is sequential based on the presence of feedback.
- 1 point for clear justification and explanation linking feedback to memory and sequential behavior.